Початок роботи з C# і ASP.NET Core в Visual Studio. Razor Pages (частина 1)

Завдання. Створіть застосунок Razor Pages за прикладом нижче наведеного Каталогу фільмів.

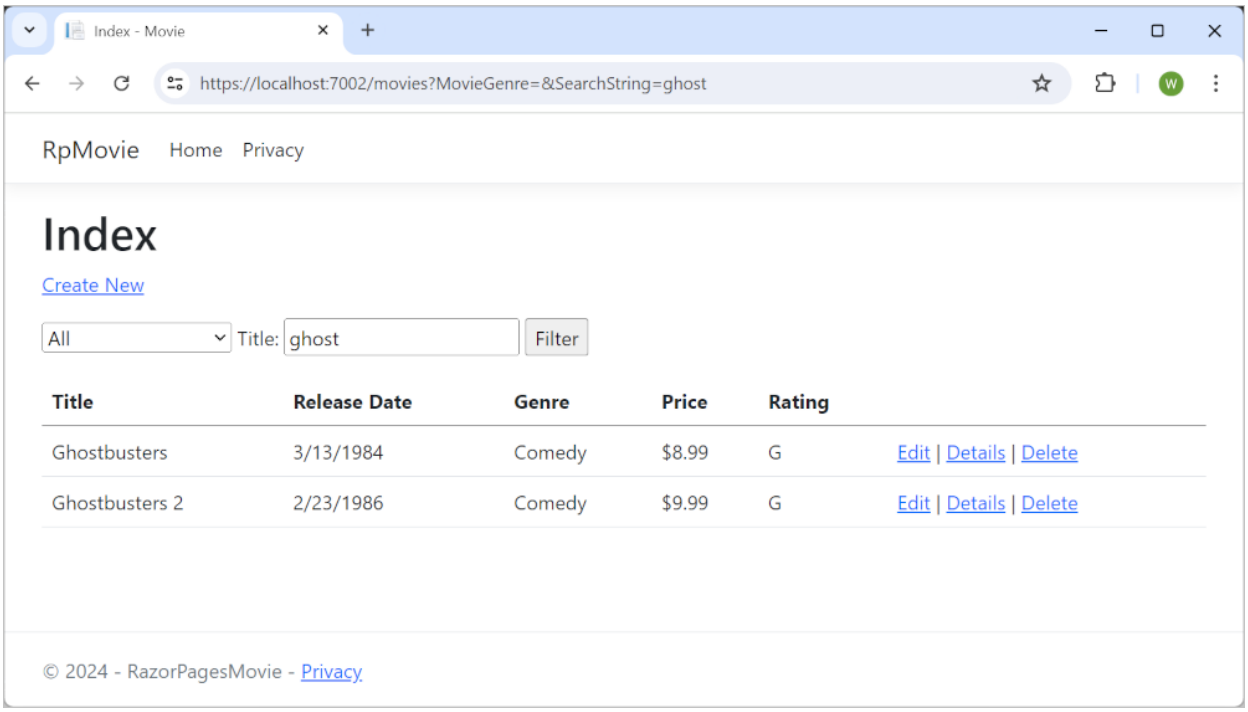
Замість каталогу фільмів, що наводиться в прикладі зберігайте каталог книг, музики, товарів, адрес і т.д. на власний вибір. **Ваша програма не повинна бути повністю ідентичною до опису в прикладі.**

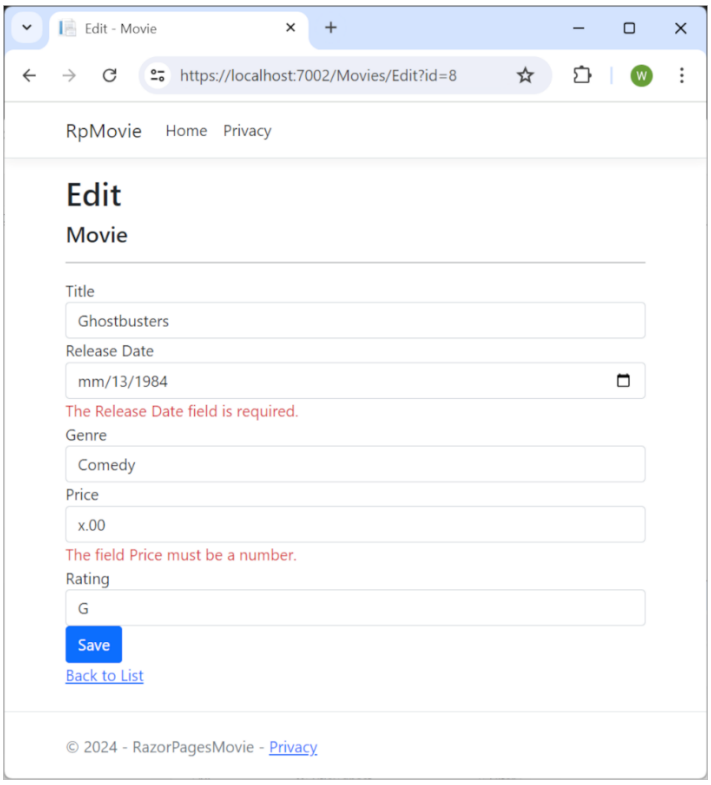
ASP.NET Core – це кроссплатформна платформа з відкритим кодом для створення додатків, підключених до Інтернету, таких як веб-додатки та служби. Додатки ASP.NET Core можуть працювати на основі .NET Core або .NET Framework. Додатки ASP.NET Core можна розробляти і запускати на різних платформах, включаючи Windows, Mac і Linux. Код ASP.NET Core відкритий для загального доступу в GitHub.

Далі розглянуто основи створення веб-програми Razor Pages.
Ця серія включає в себе наступні навчальні посібники:

- Створення веб-програми Razor Pages
- Як додати модель до програми Razor Pages
- Scaffold (генерація) Razor Pages
- Робота з базою даних
- Оновлення сторінок Razor
- Додавання пошуку
- Як додати нове поле
- Додавання валідації

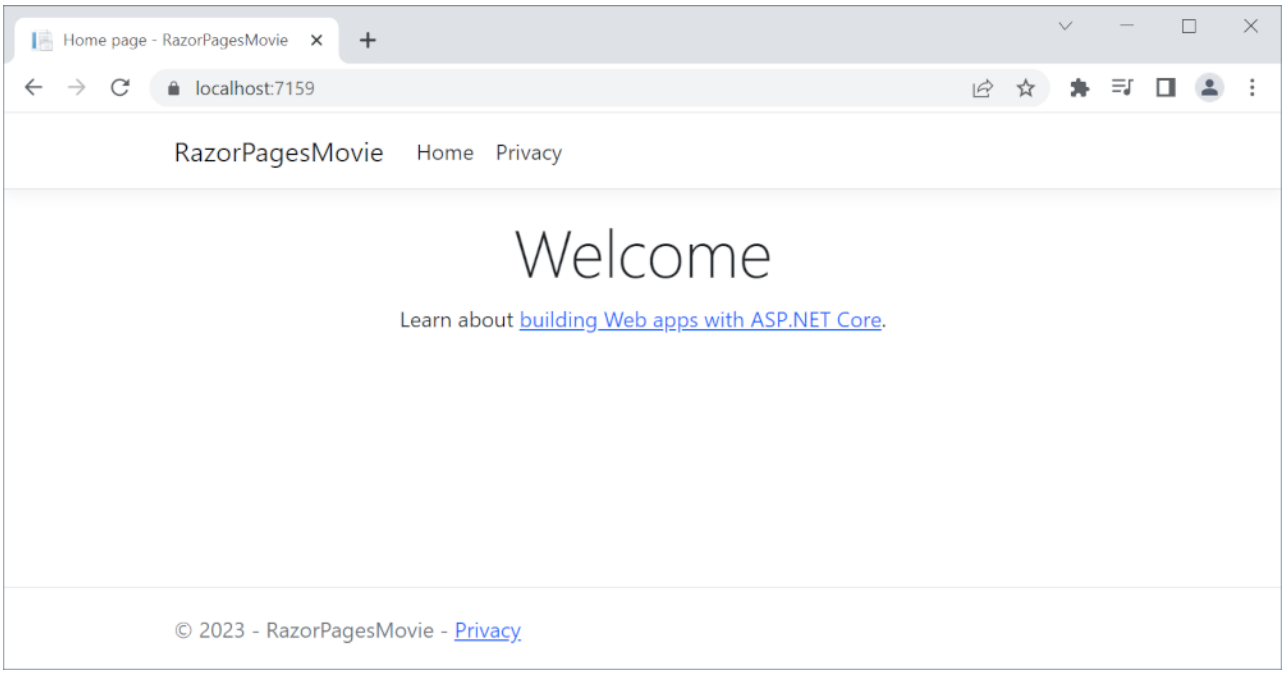
Наприкінці у вас повинна бути програма, яка дозволяє відображати базу даних фільмів і керувати нею.





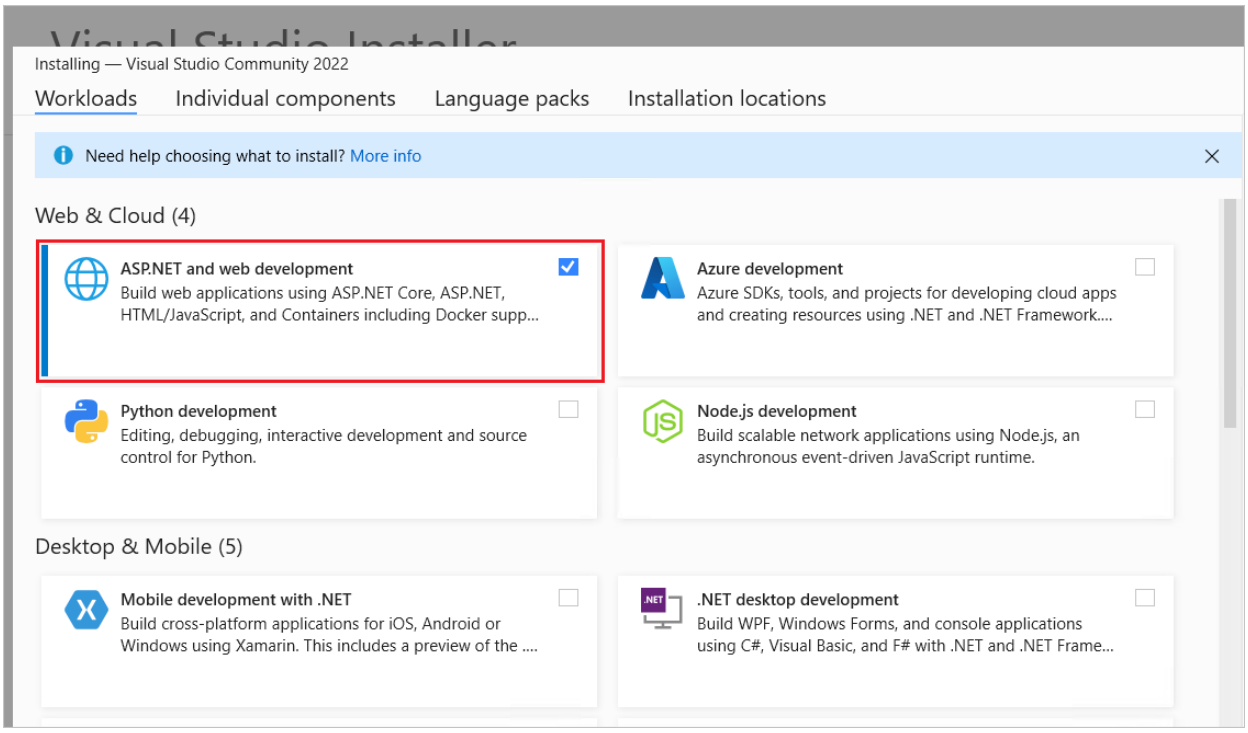
Початок роботи з Razor Pages у ASP.NET Core

В кінці цього уроку у вас буде веб-застосунок Razor Pages, який керує базою даних фільмів.



Передумови

- Visual Studio 2022 або 2026 з **робочим навантаженням на** ASP.NET та веб-розробку.



Створення веб-програми Razor Pages

Запустіть Visual Studio та виберіть **New project** (*Створити проєкт*).
У діалоговому вікні **Create a new project** (*Створення нового проєкту*) виберіть **ASP.NET Core Web App (Razor Pages)** > **Next** (*Далі*).

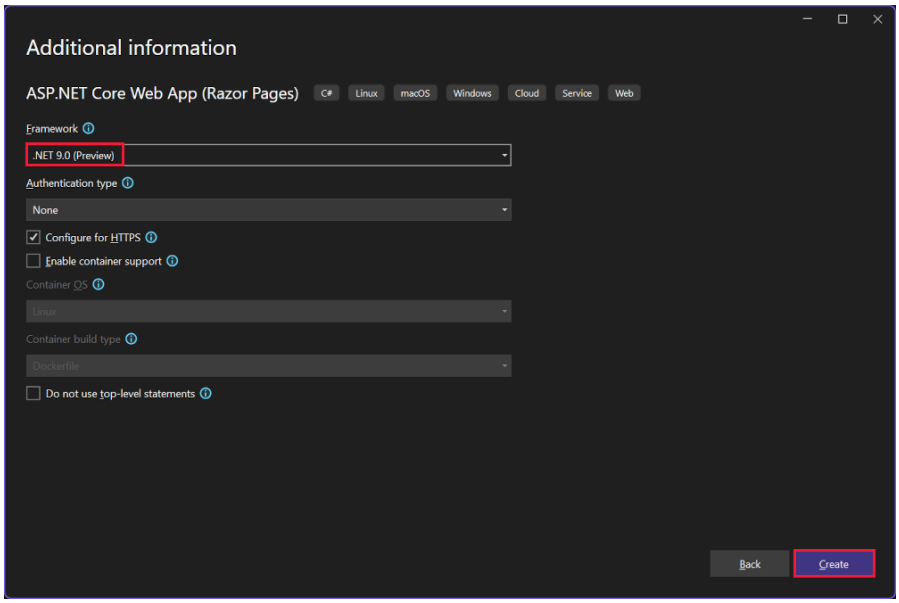
У діалоговому вікні **Configure your new project** (*Налаштування нового проєкту*) введіть **RazorPagesMovie** як ім'я проєкту. Важливо, щоб назва проєкту була саме **RazorPagesMovie**, з урахуванням регістру літер, щоб простори імен збігалися при копіюванні та вставленні прикладів коду.

Виберіть **Next** (*Далі*).

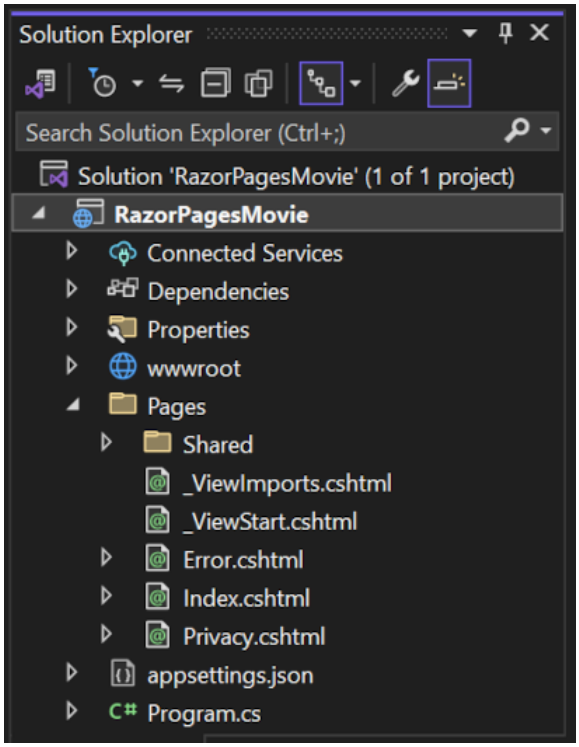
У діалоговому вікні **Additional information** (*Додаткова інформація*):

- Виберіть **.NET 9.0** (або новішу версію .Net, але можливі деякі розбіжності з нижченаведеним описом).
- Переконайтеся, що пункт **Do not use top-level statements** (*Не використовувати верхньорівневі інструкції*) **не позначений**.

Натисніть **Create** (*Створити*).



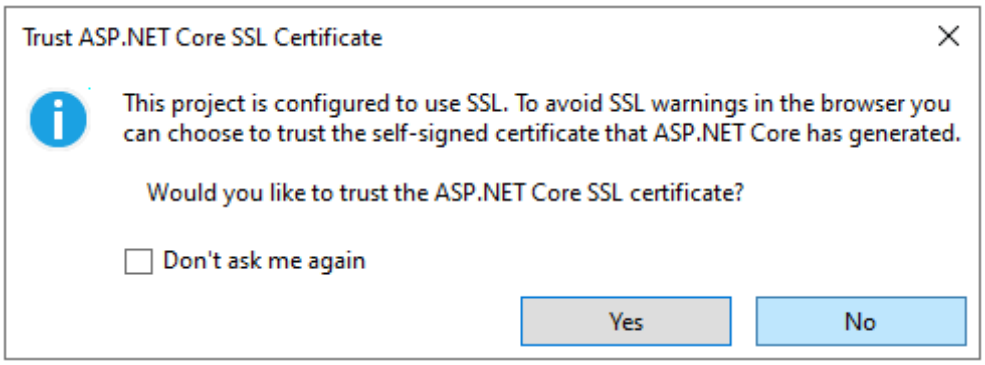
Створюється наступний стартовий проект:



Запустіть додаток

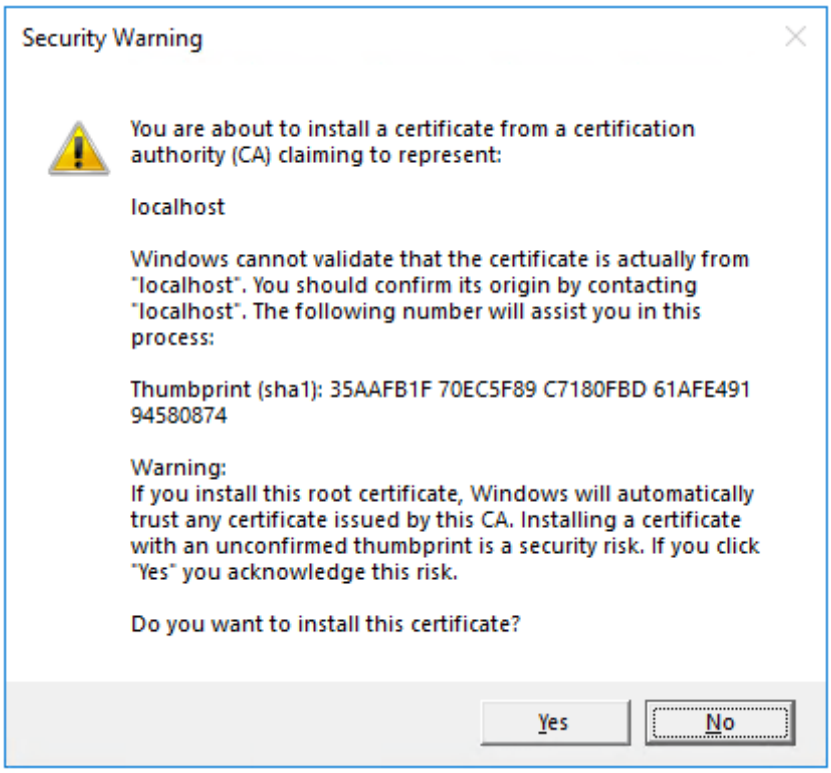
Виберіть **RazorPagesMovie** у **Провіднику рішень**, а потім натисніть Ctrl+F5, щоб запустити програму без налагоджувача.

Visual Studio відображає таке діалогове вікно, коли проект ще не налаштовано на використання SSL:



Виберіть **Так**, якщо ви довіряєте сертифікату IIS Express SSL.

Відображається таке діалогове вікно:



Виберіть **Так,** якщо ви погоджуєтеся довіряти сертифікату розробки.

Щоб дізнатися більше про довіру до браузера Firefox, перегляньте статтю Помилка сертифіката SEC_ERROR_INADEQUATE_KEY_USAGE Firefox.

Visual Studio:

- Запускає програму, яка запускає сервер Kestrel.

- Запускає браузер за замовчуванням за адресою https://localhost:<port>, який відображає інтерфейс програми. <порт> – це випадковий порт, який призначається під час створення програми.

Закрийте вікно браузера.

Перегляньте файли проекту

Наступні розділи містять огляд основних папок проектів і файлів, з якими ви будете працювати далі.

Папка Pages

Містить сторінки **Razor** та допоміжні файли. Кожна сторінка Razor – це пара файлів:

- Файл .cshtml, який має розмітку HTML із кодом C# з використанням синтаксису Razor.
- Файл .cshtml.cs із кодом C#, який обробляє події сторінки.

Допоміжні файли мають назви, які починаються з підкреслення. Наприклад, файл _Layout.cshtml налаштовує загальні для всіх сторінок елементи інтерфейсу. _Layout.cshtml налаштовує навігаційне меню у верхній частині сторінки та повідомлення про авторські права внизу сторінки. Докладнішу інформацію дивіться в розділі Компонування в ASP.NET Core.

Папка wwwroot

Містить статичні ресурси, як-от файли HTML, JavaScript і CSS. Для отримання додаткової інформації дивіться статичні файли в ASP.NET Core.

appsettings.json

Містить дані конфігурації, наприклад рядки підключення. Для отримання додаткової інформації дивіться Конфігурація в ASP.NET Core.

Program.cs

Містить наступний код:

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddRazorPages();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Error");
    // The default HSTS value is 30 days. You may want to change this for
production scenarios, see https://aka.ms/aspnetcore-hsts.
    app.UseHsts();
}

app.UseHttpsRedirection();
```

```
app.UseRouting();

app.UseAuthorization();

app.MapStaticAssets();
app.MapRazorPages();

app.Run();
```

Наступні рядки коду в цьому файлі створюють WebApplicationBuilder з попередньо налаштованими параметрами за замовчуванням, додають підтримку Razor Pages до контейнера Dependency Injection (DI) і створюють додаток:

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddRazorPages();

var app = builder.Build();
```

Сторінка винятків для розробників увімкнена за замовчуванням і надає корисну інформацію про винятки. Робочі програми не можна запускати в режимі розробки, оскільки сторінка винятків розробника може призвести до витоку конфіденційної інформації.
Наведений нижче код встановлює кінцеву точку винятку на /Error і вмикає HTTP Strict Transport Security Protocol (HSTS), коли застосунок ***не*** працює в режимі розробки:

```
// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Error");
    // The default HSTS value is 30 days. You may want to change this for
production scenarios, see https://aka.ms/aspnetcore-hsts.
    app.UseHsts();
}
```

Наприклад, попередній код запускається, коли застосунок перебуває в робочому або тестовому режимі. Щоб отримати докладнішу інформацію, перегляньте статтю Використання кількох середовищ у ASP.NET Core.
Наступний код дозволяє використовувати різне проміжне програмне забезпечення:

- app.UseHttpsRedirection(); : переспрямовує HTTP-запити на HTTPS.
- app.UseRouting(); : додає узгодження маршрутів до конвеєра проміжного програмного забезпечення. Для отримання додаткової інформації дивіться розділ Маршрутизація в ASP.NET Core.
- app.UseAuthorization(); : дозволяє користувачу отримувати доступ до безпечних ресурсів. Цей застосунок не використовує авторизацію, тому цей рядок можна видалити.
- app.MapRazorPages();: налаштовує маршрутизацію кінцевих точок для Razor Pages.

- app.MapStaticAssets(); : оптимізовує доставку статичних ресурсів у програмі, таких як HTML, CSS, зображення та JavaScript. Щоб отримати додаткові відомості, перегляньте статтю Що нового в ASP.NET Core у .NET 9.
- app.Run(); : запускає програму.

Додавання моделі до програми Razor Pages у ASP.NET Core

У цьому посібнику додані класи для управління фільмами в базі даних. Класи моделей програми використовують Entity Framework Core (EF Core) для роботи з базою даних. EF Core – це об'єктно-реляційне відображення (O/RM), яке спрощує доступ до даних. Спочатку ви пишете класи моделей, а EF Core створює базу даних.

Класи моделей відомі як класи POCO (від "**Plain-Old CLR Objects"), оскільки вони** не мають залежності від EF Core. Вони визначають властивості даних, які зберігаються в базі даних.

Додавання моделі даних

А. У **Провіднику рішень (Solution Explorer)** клацніть правою кнопкою миші *проект RazorPagesMovie* > **Add** > **New Folder**». Назвіть папку Models.

Б. Клацніть правою кнопкою миші папку Models. Виберіть **Add** > **Class.** Назвіть клас *Movie*.

В. Додайте до класу Movie такі властивості:

```
using System.ComponentModel.DataAnnotations;

namespace RazorPagesMovie.Models;

public class Movie
{
    public int Id { get; set; }
    public string? Title { get; set; }
    [DataType(DataType.Date)]
    public DateTime ReleaseDate { get; set; }
    public string? Genre { get; set; }
    public decimal Price { get; set; }
}
```

Клас «Movie» містить:

- Поле Id є обов'язковим для бази даних для первинного ключа.
- Атрибут [DataType], який визначає тип даних у властивості ReleaseDate. З цим атрибутом:
    - Користувачеві не потрібно вводити інформацію про час у поле дати.

- Відображається лише дата, а не інформація про час.
- Знак питання після рядка вказує на те, що властивість є *nullable*. Для отримання додаткової інформації дивіться статтю Nullable типи посилань.

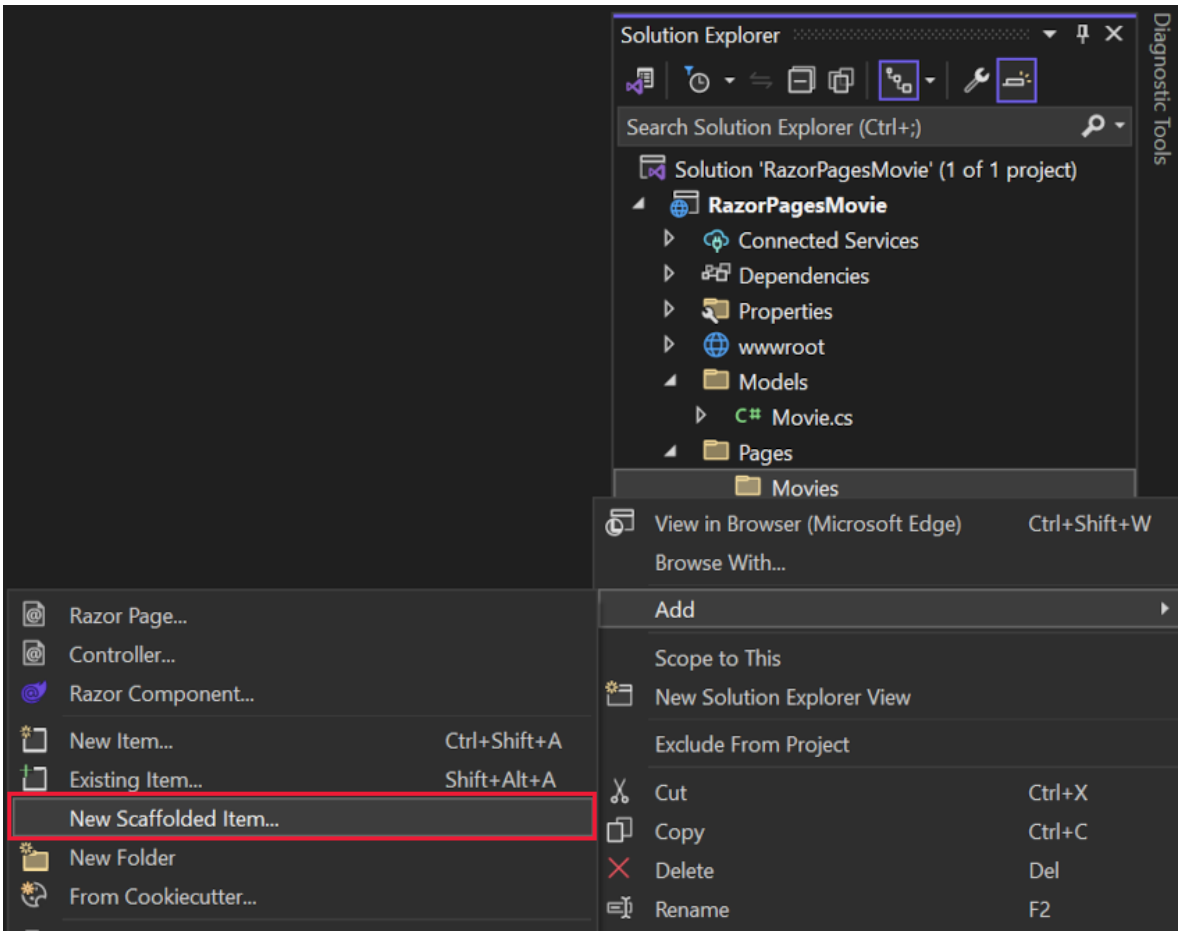
DataAnnotations розглядаються в наступному навчальному посібнику.
Зберіть проект, щоб переконатися у відсутності помилок компіляції.

Згенерувати каркас моделі Movie

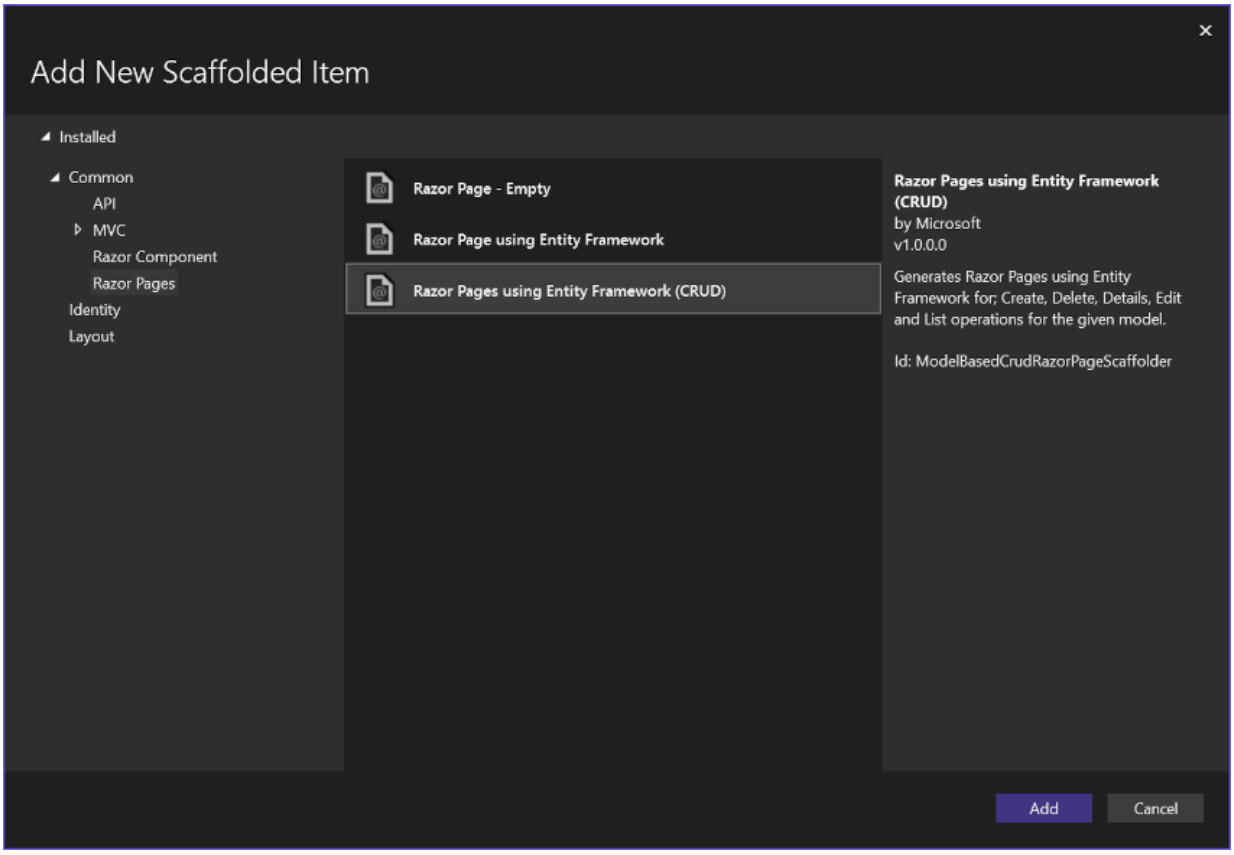
У цьому розділі модель фільму виконана генеруванням. Тобто, інструмент генерування створює сторінки для операцій створення, читання, оновлення та видалення (Create, Read, Update, Delete - CRUD) для моделі фільму.

А. Створіть папку *Pages/Movies*:
- Клацніть правою кнопкою миші папку *Pages* > **Add** > **New Folder**».
- Назвіть папку ***Movies***.

Б. Клацніть правою кнопкою миші по *папці* «Pages/Movies» > **«Add** > **New Scaffolded Item**».



В. У **діалоговому вікні** «**New Scaffolded Item**» виберіть **Razor Pages using Entity Framework (CRUD)** > **Add**.

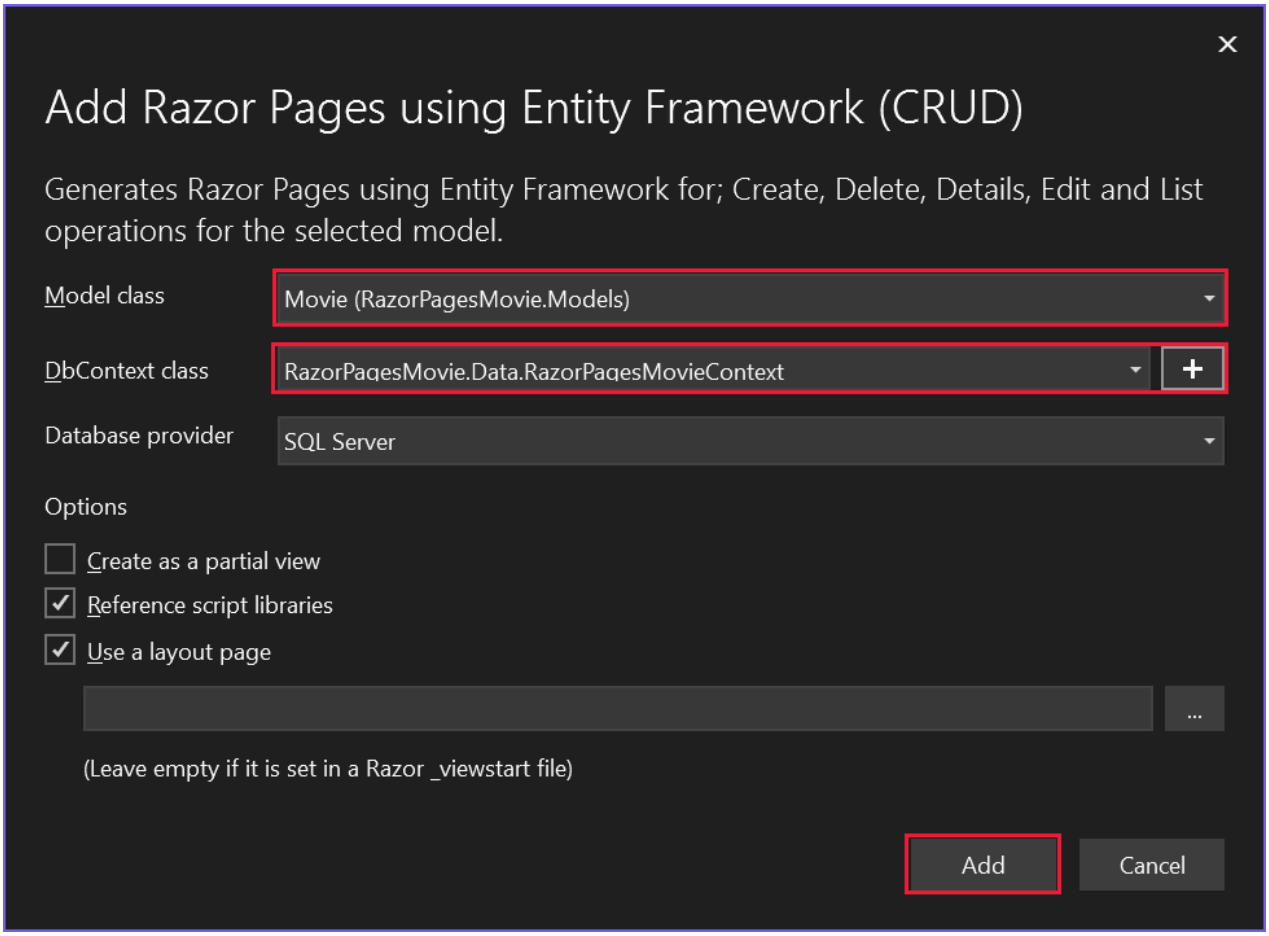


А. Завершіть діалогове вікно **Add Razor Pages using Entity Framework (CRUD):**

- У **випадаючому списку** Model Class виберіть **Movie (RazorPagesMovie.Models).**
- У **рядку** класу контексту даних **виберіть знак +** (плюс).

А. У діалоговому вікні «**Add Data Context**» генерується ім'я класу RazorPagesMovie.Data.RazorPagesMovieContext.

Б. У **розкривному списку Database provider** виберіть **SQL Server**. Виберіть **Add (Додатиі)**.



Файл appsettings.json оновлюється рядком підключення, який використовується для підключення до локальної бази даних.

Попередження

У цій статті використовується локальна база даних, яка не вимагає автентифікації користувача. У виробничих програмах має використовуватися найбезпечніший процес автентифікації. Щоб дізнатися більше про автентифікацію для розгорнутих тестових і робочих програм, перегляньте статтю **Безпечні потоки автентифікації**.

Створені та оновлені файли

У процесі риштування (Scaffolding) створюються такі файли:

- *Pages/Movies*: Create, Delete, Details, Edit та Index
- Data/RazorPagesMovieContext.cs

Про створені файли ми дізнаємося в наступному посібнику.

Процес каркаса додає до файлу Program.cs наступний виділений код :

```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using RazorPagesMovie.Data;
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddRazorPages();
builder.Services.AddDbContext<RazorPagesMovieContext>(options =>
```

```
options.UseSqlServer(builder.Configuration.GetConnectionString("RazorPage
sMovieContext") ?? throw new InvalidOperationException("Connection string
'RazorPagesMovieContext' not found.")));

var app = builder.Build();

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Error");
    // The default HSTS value is 30 days. You may want to change this for
production scenarios, see https://aka.ms/aspnetcore-hsts.
    app.UseHsts();
}

app.UseHttpsRedirection();

app.UseRouting();

app.UseAuthorization();

app.MapStaticAssets();
app.MapRazorPages();

app.Run();
```

Зміни в Program.cs пояснюються далі в цьому посібнику.

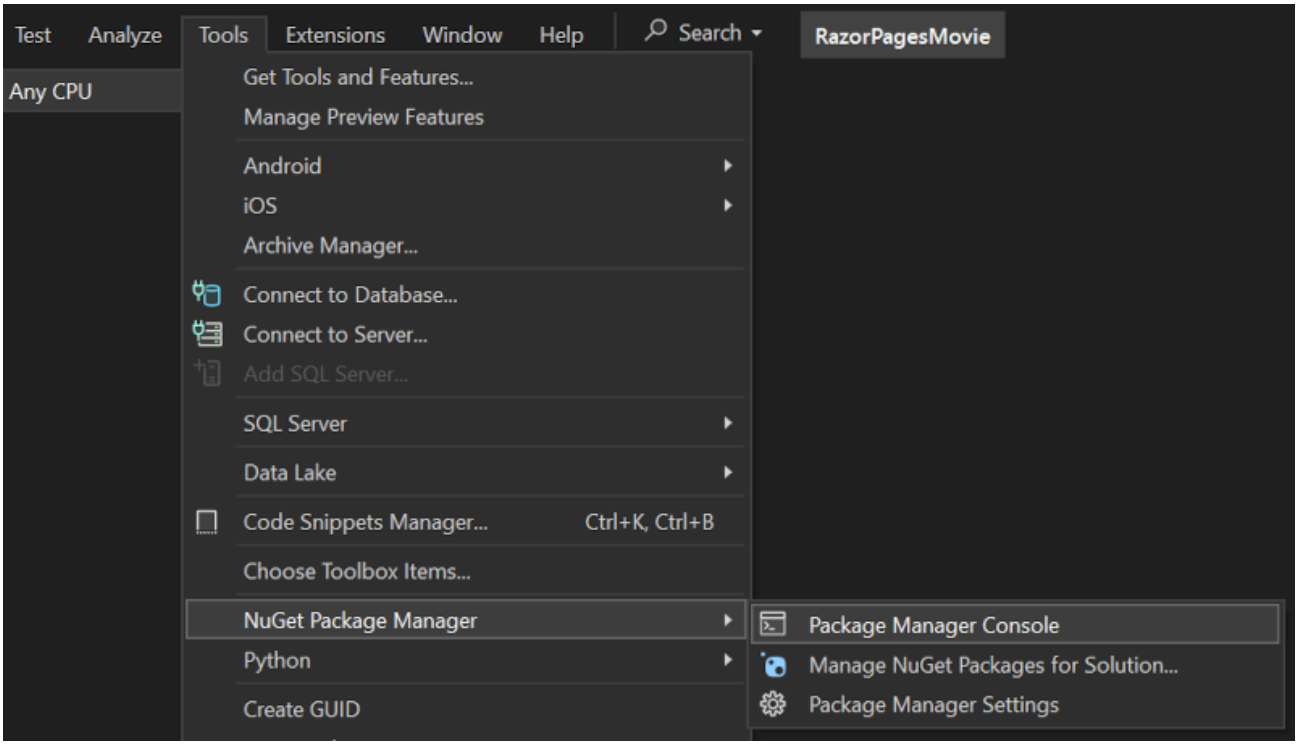
Створення початкової схеми бази даних за допомогою функції міграції EF

Функція міграції в Entity Framework Core надає можливість:

- Створити початкову схему бази даних.
- Поступово оновити схему бази даних, щоб вона синхронізувалася з моделлю даних програми. Існуючі дані в базі даних зберігаються.

У цьому розділі вікно **консолі диспетчера пакетів – Package Manager Console** (PMC) використовується для:

- Додавання початкової міграції.
- Оновлення бази даних з початковою міграцією.
- У меню **«Tools** » виберіть select **NuGet Package Manager** > **Package Manager Console**.



- У PMC вводимо наступну команду:

PowerShell

```
Add-Migration InitialCreate
```

• Команда `Add-Migration` генерує код для створення початкової схеми бази даних. Схема базується на моделі, зазначеній у `DbContext`. Аргумент `InitialCreate` використовується для назви міграції. Можна використовувати будь-яке ім'я, але за домовленістю вибирається назва, яка описує міграцію.

На дисплеї з'явиться таке попередження, яке буде вирішено на наступному кроці:

Не було вказано тип для десяткового стовпця "Ціна" у типі сутності "Фільм". Це призведе до того, що значення будуть безшумно обрізані, якщо вони не вписуються в стандартну точність і масштаб. Явно вкажіть тип стовпця SQL-сервера, який може вмістити всі значення, використовуючи 'HasColumnType()'.

- У PMC вводимо наступну команду:

PowerShell

```
Update-Database
```

Команда `Update-Database` запускає метод `Up` у міграціях, які не були застосовані. У цьому випадку команда запускає метод `Up` у файлі `Migrations/<time-stamp>_InitialCreate.cs`, який створює базу даних.

Контекст даних `RazorPagesMovieContext`:

- Походить від Microsoft.EntityFrameworkCore.DbContext.
- Визначає, які сутності включені в модель даних.
- Координує функції EF Core, такі як створення, читання, оновлення та видалення, для моделі `фільму`.

Клас RazorPagesMovieContext у згенерованому файлі Data/RazorPagesMovieContext.cs:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.EntityFrameworkCore;
using RazorPagesMovie.Models;

namespace RazorPagesMovie.Data
{
    public class RazorPagesMovieContext : DbContext
    {
        public RazorPagesMovieContext (DbContextOptions<RazorPagesMovieContext> options)
        : base(options)
        {
        }

        public DbSet<RazorPagesMovie.Models.Movie> Movie { get; set; } = default!;
    }
}
```

Попередній код створює об'єкт dbSet<Movie> для множини сутності. У термінології Entity Framework набір сутностей зазвичай відповідає таблиці бази даних. Сутність відповідає рядку в таблиці.

Ім'я рядка підключення передається в контекст шляхом виклику методу на DbContextOptions об'єкт. Для локальної розробки, Система конфігурації зчитує рядок підключення з файлу appsettings.json.

Протестуйте додаток

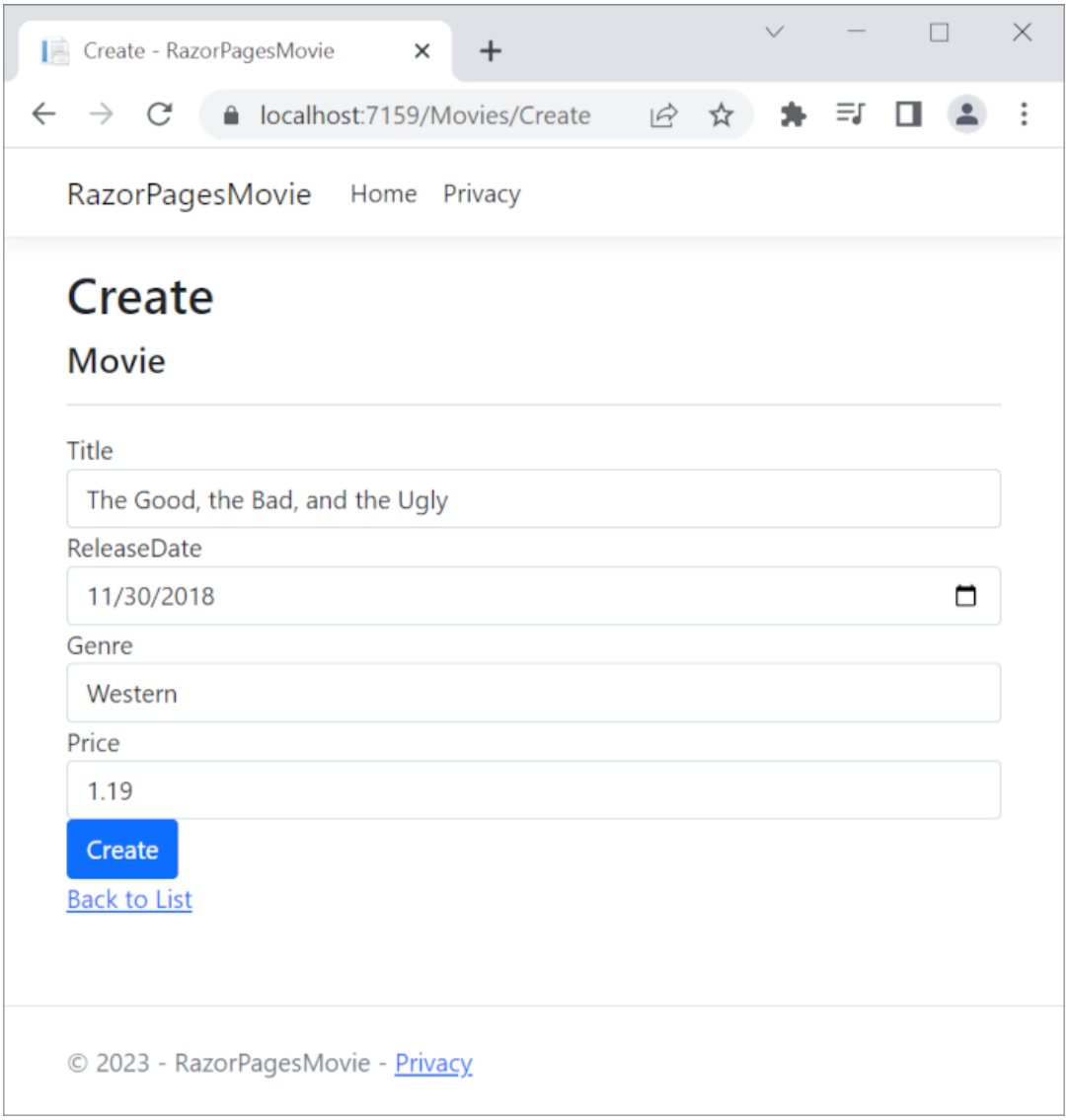
Запустіть програму та додайте /Movies до URL у браузері (http://localhost:port/movies).
Якщо з'являється таке повідомлення про помилку:

Консоль

```
SqlException: Cannot open database "RazorPagesMovieContext-GUID" requested
by the login. The login failed.
Login failed for user 'User-name'.
```

Ви пропустили етап міграції.

Перевірте посилання **Створити новий (Create New)**.



Примітка

Можливо, вам не вдасться ввести десяткові коми в поле «Ціна». Щоб підтримувати **перевірку jQuery** для неанглійських локалей, які використовують кому (",") для десяткової коми, а також для неамерикансько-англійських форматів дати, застосунок має бути глобалізований. Щоб отримати інструкції щодо глобалізації, перегляньте **цю проблему на GitHub**.

A. Перевірте посилання **«Edit»**, **«Details»** та **«Delete»**.

У наступному уроці пояснюються файли, створені за допомогою Scaffolding.

Дослідіть контекст, зареєстрований за допомогою впровадження залежностей

ASP.NET Core побудовано за допомогою впровадження залежностей. Сервіси, такі як контекст бази даних EF Core, реєструються з впровадженням залежностей під час запуску програми. Компоненти, які потребують цих сервісів (наприклад, Razor Pages), надаються за допомогою параметрів конструктора. Код конструктора, який отримує контекстний екземпляр бази даних, показаний пізніше.

Інструмент scaffolding автоматично створив контекст бази даних і зареєстрував його в контейнері впровадження залежностей. Наступний виділений код додається до файлу Program.cs за допомогою каркаса:

```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using RazorPagesMovie.Data;
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddRazorPages();
builder.Services.AddDbContext<RazorPagesMovieContext>(options =>
options.UseSqlServer(builder.Configuration.GetConnectionString("RazorPage
sMovieContext") ?? throw new InvalidOperationException("Connection string
'RazorPagesMovieContext' not found.")));

var app = builder.Build();

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Error");
    // The default HSTS value is 30 days. You may want to change this for
production scenarios, see https://aka.ms/aspnetcore-hsts.
    app.UseHsts();
}

app.UseHttpsRedirection();

app.UseRouting();

app.UseAuthorization();

app.MapStaticAssets();
app.MapRazorPages();

app.Run();
```

Згенеровані сторінки Razor в ASP.NET Core

Сторінки Create, Delete, Details і Edit

Перегляньте Pages/Movies/Index.cshtml.cs:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.EntityFrameworkCore;
using RazorPagesMovie.Data;
using RazorPagesMovie.Models;
using Microsoft.AspNetCore.Mvc.Rendering;

namespace RazorPagesMovie.Pages.Movies
{
    public class IndexModel : PageModel
    {
        private    readonly    RazorPagesMovie.Data.RazorPagesMovieContext
_context;

        public     IndexModel(RazorPagesMovie.Data.RazorPagesMovieContext
context)
        {
            _context = context;
        }

        public IList<Movie> Movie { get;set; } = default!;

        public async Task OnGetAsync()
        {
            Movie = await _context.Movie.ToListAsync();
        }
    }
}
```

Сторінки Razor є похідними PageModel. Згідно з домовленістю, похідний від `PageModel` клас має ім'я `PageNameModel`. Наприклад, сторінка `Index` має назву `IndexModel`.
Конструктор використовує ін'єкцію залежностей для додавання `RazorPagesMovieContext` на сторінку:

```
public class IndexModel : PageModel
{
    private readonly RazorPagesMovie.Data.RazorPagesMovieContext _context;

    public IndexModel(RazorPagesMovie.Data.RazorPagesMovieContext context)
    {
        _context = context;
    }
```

Дивіться Асинхронний код для отримання додаткової інформації про асинхронне програмування за допомогою Entity Framework.

Коли для сторінки робиться запит GET, метод OnGetAsync повертає список фільмів на сторінку Razor. На

сторінці Razor викликається OnGetAsync або OnGet для ініціалізації стану сторінки. У цьому випадку OnGetAsync отримує список фільмів і відображає їх.
Коли OnGet повертає void або OnGetAsync повертає Task, оператор return не використовується. Наприклад, перегляньте сторінку конфіденційності:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

namespace RazorPagesMovie.Pages
{
    public class PrivacyModel : PageModel
    {
        private readonly ILogger<PrivacyModel> _logger;

        public PrivacyModel(ILogger<PrivacyModel> logger)
        {
            _logger = logger;
        }

        public void OnGet()
        {
        }
    }

}
```

Коли тип return має значення IActionResult або Task<IActionResult>, необхідно надати return. Наприклад, метод Pages/Movies/Create.cshtml.cs OnPostAsync :

```
public async Task<IActionResult> OnPostAsync()
{
    if (!ModelState.IsValid)
    {
        return Page();
    }

    _context.Movie.Add(Movie);
    await _context.SaveChangesAsync();

    return RedirectToPage("./Index");
}
```

Перегляньте сторінку Razor Pages/Movies/Index.cshtml :

CSHTML

```
@page
@model RazorPagesMovie.Pages.Movies.IndexModel

@{
    ViewData["Title"] = "Index";
}
```

```
<h1>Index</h1>

<p>
    <a asp-page="Create">Create New</a>
</p>
<table class="table">
    <thead>
        <tr>
            <th>
                @Html.DisplayNameFor(model => model.Movie[0].Title)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Movie[0].ReleaseDate)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Movie[0].Genre)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Movie[0].Price)
            </th>
            <th></th>
        </tr>
    </thead>
    <tbody>
@foreach (var item in Model.Movie) {
        <tr>
            <td>
                @Html.DisplayFor(modelItem => item.Title)
            </td>
            <td>
                @Html.DisplayFor(modelItem => item.ReleaseDate)
            </td>
            <td>
                @Html.DisplayFor(modelItem => item.Genre)
            </td>
            <td>
                @Html.DisplayFor(modelItem => item.Price)
            </td>
            <td>
                <a asp-page="./Edit" asp-route-id="@item.Id">Edit</a> |
                <a             asp-page="./Details"             asp-route-
id="@item.Id">Details</a> |
                <a asp-page="./Delete" asp-route-id="@item.Id">Delete</a>
            </td>
        </tr>
}
    </tbody>
</table>
```

Razor може переходити з HTML у C# або в розмітку, специфічну для Razor. Коли за символом @ слідує зарезервоване ключове слово Razor, він переходить у розмітку, специфічну для Razor, в іншому випадку він переходить у C#.

Директива @page

Директива @page Razor робить файл дією MVC, що означає, що він може обробляти запити. @page має бути першою директивою Razor на сторінці. @page і @model є прикладами переходу на розмітку, специфічну для Razor. Дивіться Синтаксис Razor для отримання додаткової інформації.

Директива @model

CSHTML

```
@page
@model RazorPagesMovie.Pages.Movies.IndexModel
```

Директива @model визначає тип моделі, що передається на сторінку Razor Page. У попередньому прикладі рядок @model робить похідний клас PageModel доступним для Razor Page. Модель використовується в HTML-хелперах @Html.DisplayNameFor та @Html.DisplayFor на сторінці.
Перегляньте лямбда-вираз, який використовується в наступному HTML-помічнику:

CSHTML

```
@Html.DisplayNameFor(model => model.Movie[0].Title)
```

HTML-помічник DisplayNameFor перевіряє властивість Title, на яку посилається лямбда-вираз, щоб визначити відображуване ім'я. Перевіряється саме вираз лямбда, а не його обчислення. Це означає, що не виникає помилки доступу, якщо model, model.Movie або model.Movie[0] дорівнює null чи є порожнім. Коли ж вираз лямбда обчислюється, наприклад, у випадку з @Html.DisplayFor(modelItem => item.Title), відбувається обчислення значень властивостей моделі.

Сторінка макета (Layout page)

Виберіть пункти меню *RazorPagesMovie*, *Home* та *Privacy*. Кожна сторінка відображає однаковий макет меню. Макет меню реалізований у файлі

Pages/Shared/_Layout.cshtml.
Відкрийте та перегляньте файл **Pages/Shared/_Layout.cshtml**.
Шаблони макетів дозволяють розміщення контейнера HTML:

- Задавати в одному місці.
- Використовувати на кількох сторінках сайту.

Знайдіть рядок **@RenderBody()**.
RenderBody є заповнювачем, у який підставляються всі представлення, специфічні для конкретної сторінки, обгорнуті у сторінку-макет.
Наприклад, якщо вибрати посилання *Privacy*, то представлення **Pages/Privacy.cshtml** буде відображене всередині методу **RenderBody**.

ViewData та компонування

Розгляньте наступну розмітку з файлу Pages/Movies/Index.cshtml:

CSHTML
```
@page
@model RazorPagesMovie.Pages.Movies.IndexModel

@{
    ViewData["Title"] = "Index";
}
```

Попередньо виділена розмітка є прикладом переходу Razor до C#. Символи { і } охоплюють блок коду C#.
Базовий клас **PageModel** містить властивість **ViewData** типу словника (dictionary), яку можна використовувати для передачі даних у представлення (View). Об'єкти додаються до словника **ViewData** за допомогою шаблону «ключ–значення». У наведеному вище прикладі властивість **Title** додається до словника **ViewData**.
Властивість **Title** використовується у файлі **Pages/Shared/_Layout.cshtml**. Нижче показано перші кілька рядків розмітки з файлу **_Layout.cshtml**.

CSHTML
```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0"
/>
    <title>@ViewData["Title"] - RazorPagesMovie</title>
    <link  rel="stylesheet"  href="~/lib/bootstrap/dist/css/bootstrap.css"
/>
    <link     rel="stylesheet"     href="~/css/site.css"     asp-append-
version="true" />
    <link   rel="stylesheet"   href="~/RazorPagesMovie.styles.css"   asp-
append-version="true" />
```

Оновлення макета

Змініть елемент <title> у файлі Pages/Shared/_Layout.cshtml так, щоб він відображав *Movie*, а не *RazorPagesMovie*.
```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0"
/>
    <title>@ViewData["Title"] - Movie</title>
```

2. **Знайдіть наступний елемент посилання у файлі Pages/Shared/_Layout.cshtml.**

```
<a class="navbar-brand" asp-area="" asp-page="/Index">RazorPagesMovie</a>
```

3. **Замініть попередній елемент наступною розміткою:**

```
<a class="navbar-brand" asp-page="/Movies/Index">RazorPagesMovie</a>
```

Попередній елемент посилання є **Tag Helper**. У цьому випадку це **Anchor Tag Helper**. Атрибут і значення asp-page="/Movies/Index" створює посилання на сторінку Razor **/Movies/Index**. Атрибут asp-area має порожнє значення, тому в посиланні область (area) не використовується. Детальніше див. розділ **Areas**.

4. **Збережіть зміни та протестуйте застосунок, вибравши посилання RpMovie.** Якщо виникнуть проблеми, перегляньте файл **_Layout.cshtml** у GitHub.
5. **Протестуйте посилання Home, RpMovie, Create, Edit і Delete.** Кожна сторінка задає власний заголовок, який ви можете бачити на вкладці браузера. Якщо додати сторінку в закладки, її заголовок буде використано як назву закладки.

Примітка
Можливо, ви не зможете вводити десяткові коми у полі *Price*. Щоб підтримати jQuery-валідацію для неангломовних локалей, які використовують кому (,) як десятковий роздільник, а також для форматів дат, відмінних від американських, потрібно зробити кроки для глобалізації застосунку. Див. **GitHub issue 4076** з інструкціями щодо додавання підтримки десяткової коми.

Властивість Layout задається у файлі **Pages/_ViewStart.cshtml**:

```
@{
    Layout = "_Layout";
}
```

Попередня розмітка встановлює файл макета на `Pages/Shared/_Layout.cshtml` для всіх файлів Razor у папці *Pages*. Дивіться Макет для отримання додаткової інформації.

Модель «Створити сторінку»

Перегляньте модель сторінки `Pages/Movies/Create.cshtml.cs` :

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.AspNetCore.Mvc.Rendering;
using RazorPagesMovie.Data;
using RazorPagesMovie.Models;

namespace RazorPagesMovie.Pages.Movies
{
    public class CreateModel : PageModel
    {
```

```
        private readonly RazorPagesMovie.Data.RazorPagesMovieContext
_context;

        public CreateModel(RazorPagesMovie.Data.RazorPagesMovieContext
context)
        {
            _context = context;
        }

        public IActionResult OnGet()
        {
            return Page();
        }

        [BindProperty]
        public Movie Movie { get; set; } = default!;

        // To protect from overposting attacks, see
https://aka.ms/RazorPagesCRUD
        public async Task<IActionResult> OnPostAsync()
        {
            if (!ModelState.IsValid)
            {
                return Page();
            }

            _context.Movie.Add(Movie);
            await _context.SaveChangesAsync();

            return RedirectToPage("./Index");
        }
    }
}
```

Метод **OnGet** ініціалізує будь-який стан, необхідний для сторінки. Для сторінки **Create** не потрібно нічого ініціалізувати, тому повертається **Page**. Пізніше в цьому посібнику буде показано приклад, де **OnGet** виконує ініціалізацію стану. Метод **Page** створює об'єкт **PageResult**, який відображає сторінку **Create.cshtml**.

Властивість **Movie** використовує атрибут **[BindProperty]**, щоб явно дозволити прив'язування моделі (*opt-in model binding*). Коли форма **Create** надсилає значення, середовище виконання ASP.NET Core прив'язує надіслані дані до моделі **Movie**.

Метод **OnPostAsync** виконується тоді, коли сторінка надсилає дані форми:

```
public async Task<IActionResult> OnPostAsync()
{
    if (!ModelState.IsValid)
    {
        return Page();
    }

    _context.Movie.Add(Movie);
```

```
    await _context.SaveChangesAsync();

    return RedirectToPage("./Index");
}
```

У разі виявлення будь-яких помилок моделі форма відображається знову разом із будь-якими опублікованими даними форми. Більшість помилок моделі можуть бути виявлені на стороні клієнта до того, як форма буде опублікована. Прикладом помилки моделі є публікація значення в полі дати, яке не можна перетворити на дату. Валідація на стороні клієнта та валідація моделі обговорюються далі в навчальному посібнику.
Якщо помилок моделі немає:

- Дані зберігаються.
- Браузер перенаправляється на сторінку Індекс.

Створення Razor Page

Перегляньте Pages/Movies/Create.cshtml

CSHTML

```
@page
@model RazorPagesMovie.Pages.Movies.CreateModel

@{
    ViewData["Title"] = "Create";
}

<h1>Create</h1>

<h4>Movie</h4>
<hr />
<div class="row">
    <div class="col-md-4">
        <form method="post">
            <div asp-validation-summary="ModelOnly" class="text-danger"></div>
            <div class="form-group">
                <label asp-for="Movie.Title" class="control-label"></label>
                <input asp-for="Movie.Title" class="form-control" />
                <span asp-validation-for="Movie.Title" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Movie.ReleaseDate" class="control-label"></label>
                <input asp-for="Movie.ReleaseDate" class="form-control" />
                <span     asp-validation-for="Movie.ReleaseDate"     class="text-
danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Movie.Genre" class="control-label"></label>
                <input asp-for="Movie.Genre" class="form-control" />
                <span asp-validation-for="Movie.Genre" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Movie.Price" class="control-label"></label>
                <input asp-for="Movie.Price" class="form-control" />
                <span asp-validation-for="Movie.Price" class="text-danger"></span>
```

```
            </div>
            <div class="form-group">
                <input type="submit" value="Create" class="btn btn-primary" />
            </div>
        </form>
    </div>
</div>

<div>
    <a asp-page="Index">Back to List</a>
</div>

@section Scripts {
    @{await Html.RenderPartialAsync("_ValidationScriptsPartial");}
}
```

Visual Studio відображає такі теги характерним жирним шрифтом, який використовується для допоміжних тегів:

- <form method="post">
-
- <label asp-for="Movie.Title" class="control-label"></label>
- <вхід asp-for="Movie.Title" class="form-control" />
-

Create.cshtml

```
@page
@model RazorPagesMovie.Pages.Movies.CreateModel

@{
    ViewData["Title"] = "Create";
}

<h1>Create</h1>

<h4>Movie</h4>
<hr />
<div class="row">
    <div class="col-md-4">
        <form method="post">
            <div asp-validation-summary="ModelOnly" class="text-danger"></div>
            <div class="form-group">
                <label asp-for="Movie.Title" class="control-label"></label>
                <input asp-for="Movie.Title" class="form-control" />
                <span asp-validation-for="Movie.Title" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Movie.ReleaseDate" class="control-label"></label>
                <input asp-for="Movie.ReleaseDate" class="form-control" />
                <span asp-validation-for="Movie.ReleaseDate" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Movie.Genre" class="control-label"></label>
                <input asp-for="Movie.Genre" class="form-control" />
                <span asp-validation-for="Movie.Genre" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Movie.Price" class="control-label"></label>
                <input asp-for="Movie.Price" class="form-control" />
                <span asp-validation-for="Movie.Price" class="text-danger"></span>
            </div>
            <div class="form-group">
                <input type="submit" value="Create" class="btn btn-primary" />
            </div>
```

Елемент `<form method="post">` Помічник тегів форми. Помічник (хелпер) тегів форми автоматично включає antiforgery token.
Двигун генерації сторінок створює розмітку Razor для кожного поля в моделі, крім ID, подібно до наступного:

CSHTML

```
<div asp-validation-summary="ModelOnly" class="text-danger"></div>
<div class="form-group">
    <label asp-for="Movie.Title" class="control-label"></label>
    <input asp-for="Movie.Title" class="form-control" />
    <span asp-validation-for="Movie.Title" class="text-danger"></span>
</div>
```

Тег-помічники валідації (<div asp-validation-summary> і <span asp-validation-for>) відображають повідомлення про помилки валідації. Тема валідації буде розглянута детальніше пізніше в цьому курсі.

Label Tag Helper

<label asp-for="Movie.Title" class="control-label"></label>

Генерує підпис (caption) і атрибут [for] для властивості **Title**.

Input Tag Helper

<input asp-for="Movie.Title" class="form-control">

Використовує атрибути **DataAnnotations** і створює необхідні HTML-атрибути для роботи jQuery Validation на клієнтській стороні.

Щоб дізнатися більше про тег-помічники, зокрема про <form method="post">, див. **Tag Helpers в ASP.NET Core**.

Об'єкт `RazorPagesMovieContext` виконує завдання підключення до бази даних та відображення об'єктів Movie до записів бази даних. Контекст бази даних реєструється в контейнері Dependency Injection в `Program.cs`:

```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using RazorPagesMovie.Data;
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddRazorPages();
builder.Services.AddDbContext<RazorPagesMovieContext>(options =>

options.UseSqlServer(builder.Configuration.GetConnectionString("RazorPage
```

```
sMovieContext") ?? throw new InvalidOperationException("Connection string
'RazorPagesMovieContext' not found.")));
```

Система конфігурації ASP.NET Core зчитує ключ `ConnectionString`. Для локальної розробки, конфігурація отримує рядок підключення з файлу `appsettings.json`.

Згенерований рядок підключення схожий на наступний JSON:

JSON

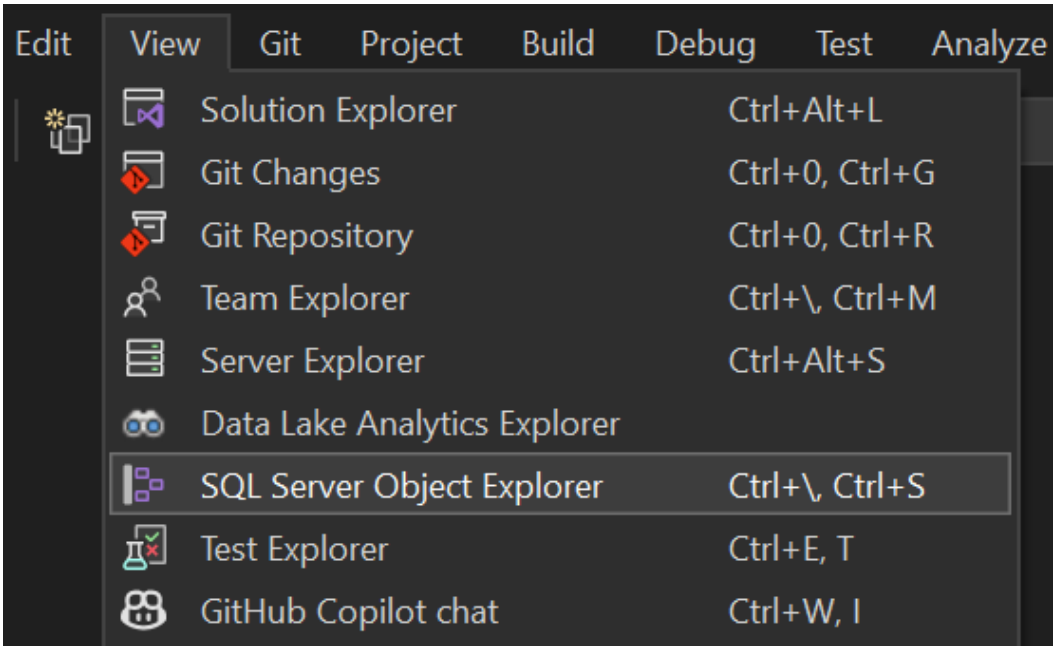
```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "RazorPagesMovieContext":
"Server=(localdb)\\mssqllocaldb;Database=RazorPagesMovieContext-f2e0482c-
952d-4b1c-afe9-
a1a3dfe52e55;Trusted_Connection=True;MultipleActiveResultSets=true"
  }
}
```

Попередження
У цьому уроці використовується локальна база даних, яка не вимагає автентифікації користувача. У виробничих програмах має використовуватися найбезпечніший процес автентифікації. Щоб дізнатися більше про автентифікацію для розгорнутих тестових і робочих програм, перегляньте статтю **Безпечні потоки автентифікації**.

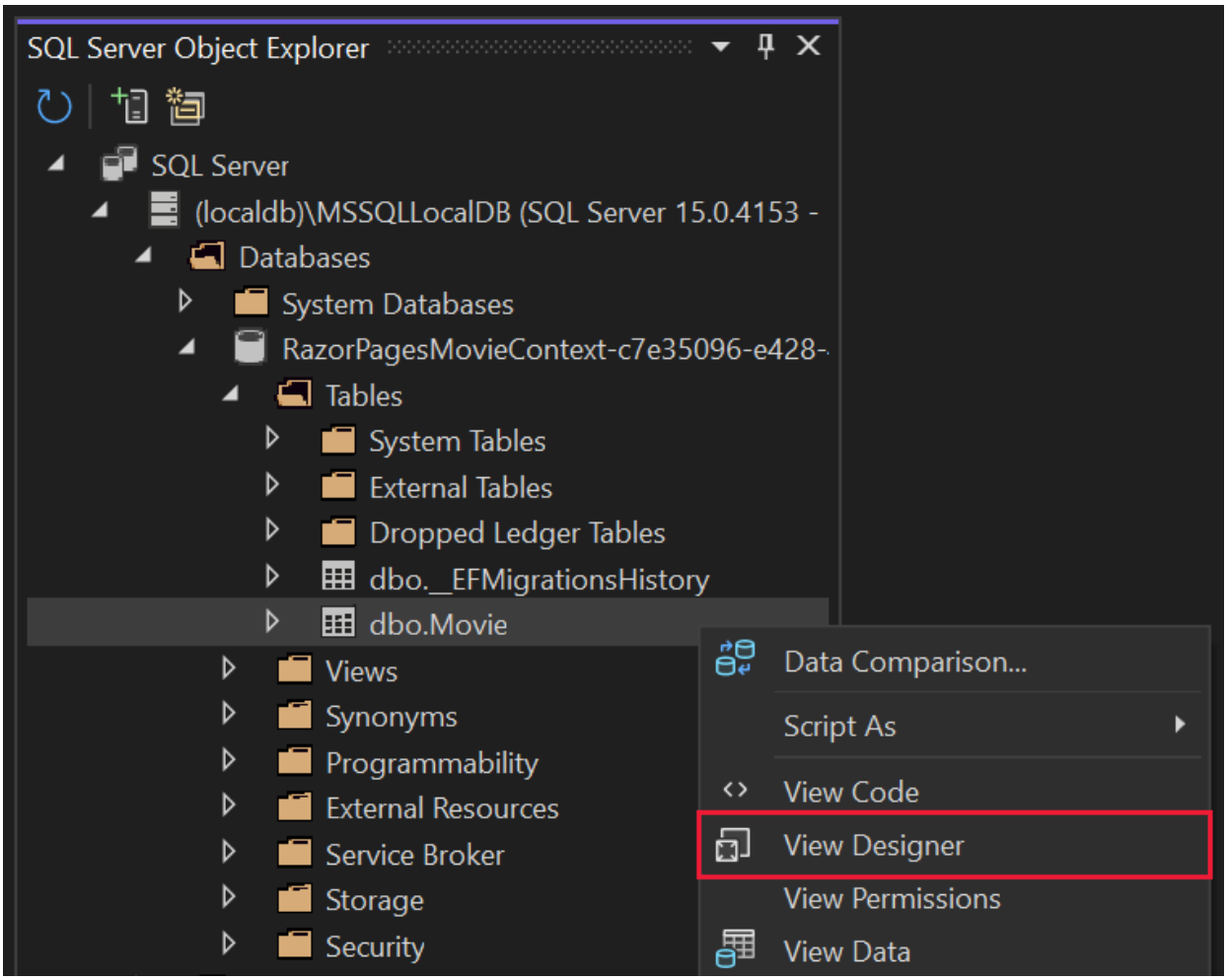
SQL Server Express LocalDB

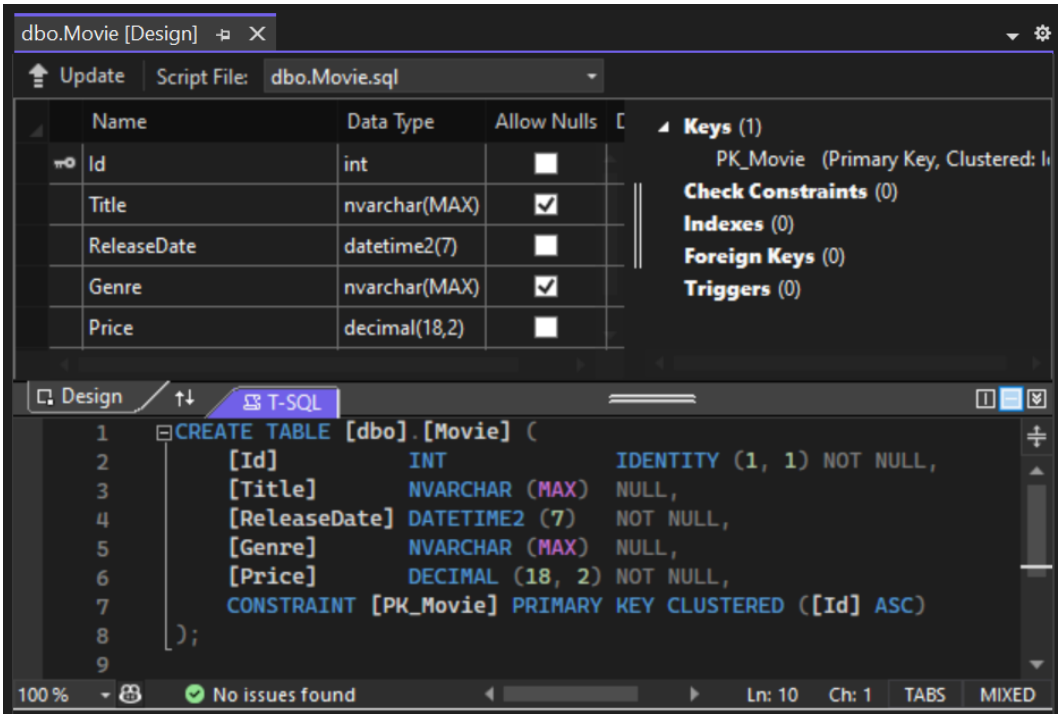
LocalDB — це полегшена версія движка баз даних SQL Server Express, призначена для розробки програм. LocalDB запускається на вимогу і працює в режимі користувача, тому немає складної конфігурації. За замовчуванням база даних LocalDB створює файли `*.mdf` в директорії C:\Users\<user>\.

У меню **View** відкрийте **SQL Server Object Explorer** (SSOX).



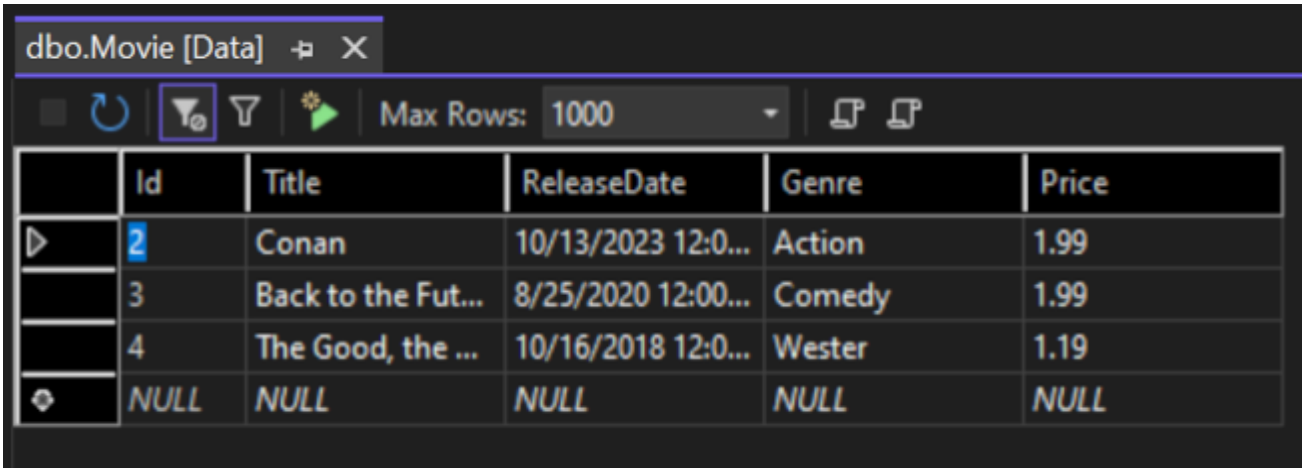
Клацніть правою кнопкою миші по таблиці «**Movie** » і виберіть «**View Designer**»:





Зверніть увагу на значок ключа поруч із `ID`. За замовчуванням EF створює властивість з іменем `ID` для первинного ключа.

Клацніть правою кнопкою миші по `таблиці «Фільм»` і виберіть **«Переглянути дані»**:



Наповнення бази даних

Створіть новий клас з іменем `SeedData` у папці Models з наступним кодом:

```
using Microsoft.EntityFrameworkCore;
using RazorPagesMovie.Data;

namespace RazorPagesMovie.Models;
```

```
public static class SeedData
{
    public static void Initialize(IServiceProvider serviceProvider)
    {
        using (var context = new RazorPagesMovieContext(
            serviceProvider.GetRequiredService<
                DbContextOptions<RazorPagesMovieContext>>()))
        {
            if (context == null || context.Movie == null)
            {
                throw new ArgumentNullException("Null
RazorPagesMovieContext");
            }

            // Look for any movies.
            if (context.Movie.Any())
            {
                return;   // DB has been seeded
            }

            context.Movie.AddRange(
                new Movie
                {
                    Title = "When Harry Met Sally",
                    ReleaseDate = DateTime.Parse("1989-2-12"),
                    Genre = "Romantic Comedy",
                    Price = 7.99M
                },

                new Movie
                {
                    Title = "Ghostbusters ",
                    ReleaseDate = DateTime.Parse("1984-3-13"),
                    Genre = "Comedy",
                    Price = 8.99M
                },

                new Movie
                {
                    Title = "Ghostbusters 2",
                    ReleaseDate = DateTime.Parse("1986-2-23"),
                    Genre = "Comedy",
                    Price = 9.99M
                },

                new Movie
                {
                    Title = "Rio Bravo",
                    ReleaseDate = DateTime.Parse("1959-4-15"),
                    Genre = "Western",
                    Price = 3.99M
                }
```

```
            );
            context.SaveChanges();
        }
    }
}
```

Якщо в базі даних є фільми, повертається початковий ініціалізатор, і фільми не додаються.

```
if (context.Movie.Any())
{
    return;
}
```

Додайте початковий ініціалізатор

Оновіть Program.cs наступним виділеним кодом:

```
using Microsoft.EntityFrameworkCore;
using RazorPagesMovie.Data;
using RazorPagesMovie.Models;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddRazorPages();
builder.Services.AddDbContext<RazorPagesMovieContext>(options =>

options.UseSqlServer(builder.Configuration.GetConnectionString("RazorPage
sMovieContext") ?? throw new InvalidOperationException("Connection string
'RazorPagesMovieContext' not found.")));

var app = builder.Build();

using (var scope = app.Services.CreateScope())
{
    var services = scope.ServiceProvider;

    SeedData.Initialize(services);
}

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();

app.UseRouting();
```

```
app.UseAuthorization();

app.MapStaticAssets();
app.MapRazorPages();

app.Run();
```

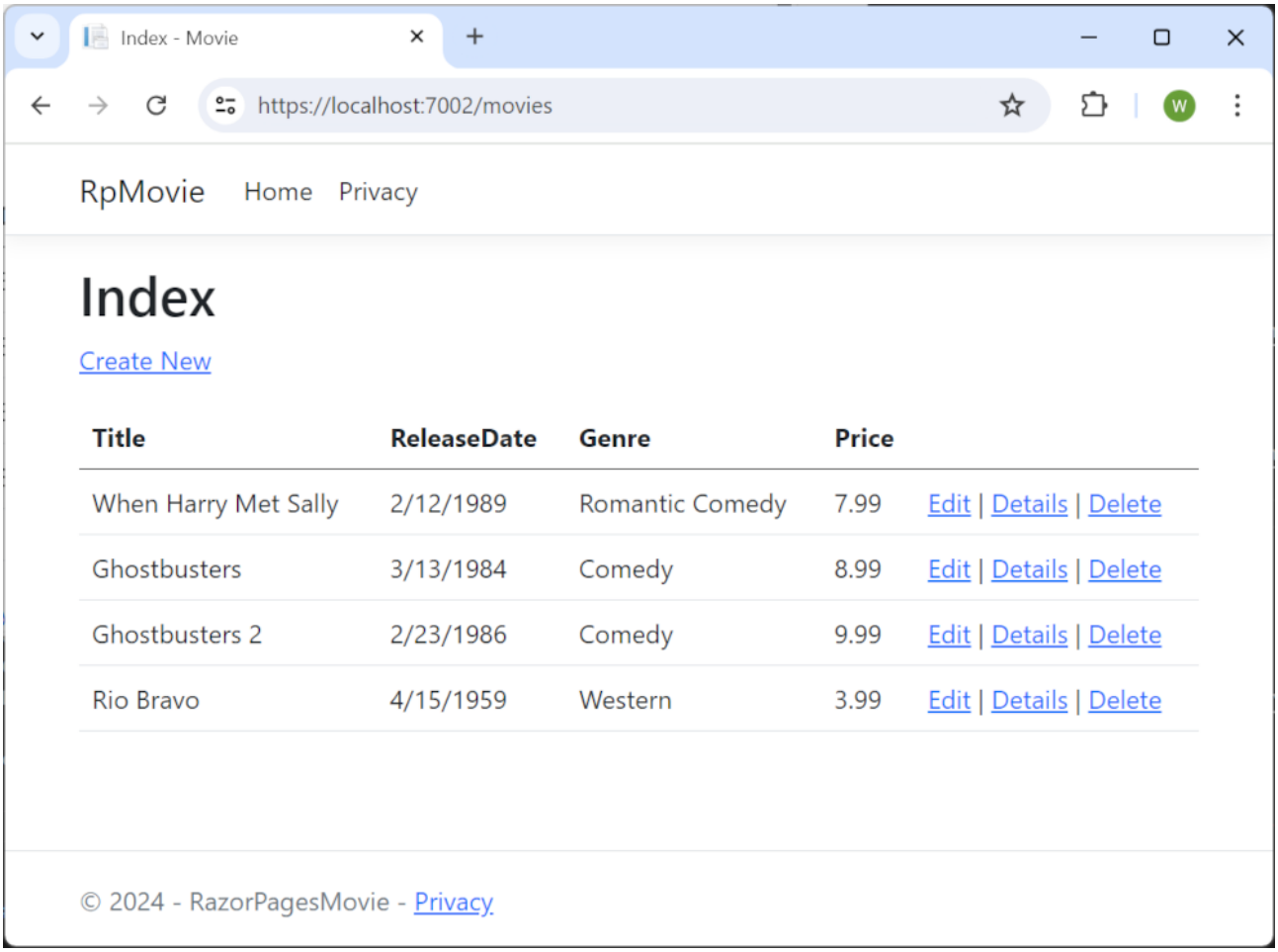
У наведеному вище коді файл **Program.cs** було змінено, щоб виконувати таке:

- Отримати екземпляр контексту бази даних із контейнера **впровадження залежностей (DI)**.
- Викликати метод **seedData.Initialize**, передавши йому екземпляр контексту БД.
- Звільнити контекст після завершення методу ініціалізації (seed). Оператор **using** гарантує, що контекст буде утилізовано.

Наступний виняток виникає, якщо **Update-Database** не було запущено:
SqlException: *Cannot open database "RazorPagesMovieContext-" requested by the login. The login failed. Login failed for user 'user name'.*
(*Неможливо відкрити базу даних "RazorPagesMovieContext-", запитану під час входу. Вхід не виконано. Помилка входу для користувача 'user name'.*)

Протестуйте додаток

Тестування застосунку
Видаліть усі записи з бази даних, щоб запустився метод наповнення (seed). Зупиніть і знову запустіть застосунок, щоб виконати наповнення бази. Якщо база даних не наповнюється, поставте брейкпоінт на рядок if (context.Movie.Any()) і пройдіться по коду крок-за-кроком.
Застосунок відобразить початкові (seeded) дані.



Використані джерела

Tutorial: Create a Razor Pages web app with ASP.NET Core. URL: https://learn.microsoft.com/uk-ua/aspnet/core/tutorials/razor-pages/?view=aspnetcore-9.0